МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Чувашский государственный университет им. И.Н. Ульянова»
Факультет информатики и вычислительной техники
Кафедра вычислительной техники

Лабораторная работа

«Проектирование веб-API»

по дисциплине «Технология разработки программного обеспечения»

Выполнил: студент группы ИВТ-42-23
Иванов И.Л.

Проверил: кандидат технических наук
Ржавин В. В.

Чебоксары, 2026

1. Описание предметной области

Объектом проектирования является информационная система «Экскурсионное бюро», разработанная в предыдущих лабораторных работах. Система автоматизирует каталог маршрутов и экскурсий, оформление и оплату договоров, выдачу электронных билетов и пакета документов, проведение экскурсий, отчётность гидов, отзывы клиентов, управление справочниками и аналитику для руководителя.

Клиентские приложения (сайт и мобильное приложение клиента, мобильное рабочее место гида, АРМ руководителя) взаимодействуют с серверной частью только через REST API. API является контрактом: сначала фиксируется спецификация, затем реализация; при расхождении изменяется реализация, а не контракт (подход API-first).

1.1. Назначение и границы API

API охватывает часть системы, связанную с публичным каталогом, оформлением и оплатой договоров, электронными билетами и документами, жизненным циклом экскурсии, отчётами гидов, справочниками руководителя и аналитикой. Базовый URL — **https://api.excursion-bureau.ru/api/v1**; версия API вынесена в путь (/api/v1).

Внешние системы не реализуются в рамках API, но используются через интеграции: «Банковский эквайринг» — для проведения платежей и возвратов, «Почтовый шлюз (SMTP)» — для отправки уведомлений и копий пакетов документов. Форматы данных: JSON (UTF-8) для всех ресурсов, PDF — для документов и посадочных ведомостей, XLSX — для выгрузки аналитических отчётов, multipart/form-data — для загрузки фотографий к отчёту.

API не покрывает внутренние операции, не видимые потребителю: структуру базы данных, внутренние сервисы, очереди сообщений, фоновые

задания и алгоритмы расчёта. Эти детали скрыты за ресурсами и действиями API.

1.2. Пользователи и роли

Пользователи API соответствуют актёрам диаграмм вариантов использования, разработанных в лабораторной работе «Модель анализа»:

– **Клиент** — просматривает каталог, оформляет и оплачивает договоры, получает электронные билеты и документы, просматривает историю, отказывается от договора, оставляет отзывы. Каталог доступен без авторизации; при оформлении договора сервер выдаёт клиентский токен (заголовок X-Client-Token), которым подписываются запросы к собственным договорам и документам.

– **Гид** — авторизуется по логину и паролю (JWT, роль GUIDE), просматривает расписание, управляет статусом занятости, начинает и завершает экскурсию, отмечает явку по билетам, формирует отчёт, фиксирует инциденты и прикрепляет фотографии, редактирует контактные данные.

– **Руководитель (Администратор)** — авторизуется (JWT, роль MANAGER), управляет каталогом маршрутов и экскурсий, автопарком, учётными записями гидов, контролирует реестр договоров и платежей, инициирует возвраты, формирует отчёты и выгружает их в Excel.

– **Сотрудник бюро** — абстрактный актёр; его права наследуются Гидом и Руководителем (вариант использования UC-32 «Авторизоваться в системе»).

Разграничение прав выполняется на уровне эндпоинтов: клиентские операции доступны по клиентскому токену, операции гида — роли GUIDE, операции руководителя — роли MANAGER. Публичные операции (каталог, отзывы) доступны без токена.

1.3. Ресурсы API и связь с моделью предметной области

Ресурсы API спроектированы по классам модели предметной области и диаграмме вариантов использования. Соответствие классов и ресурсов приведено в таблице 1.

Класс модели	Ресурс API	URL-шаблон	Назначение
ФизическоеЛицо (абстрактный)	—	/clients, /guides	Общие персональные данные клиента и гида
Клиент	client	/clients	Учёт клиентов, история договоров
Гид	guide	/guides	Профили гидов, занятость, расписание
Маршрут	route	/routes	Каталог маршрутов
Точка	point	/routes/{routeId}/points	Точки маршрута и порядок посещения
Экскурсия	excursion	/excursions	Конкретные проведения маршрута
Транспорт	transport	/transports	Автопарк и занятость транспорта
Договор	contract	/contracts	Оформление, оплата и состояние заказа
Платёж	payment	/payments	Платежи, статусы и возвраты
ЭлектронныйБилет	ticket	/tickets	Билеты участников и отметки явки
ОтчётОПроведении	report	/reports	Отчёты гидов о проведении
Отзыв	review	/excursions/{id}/reviews	Отзывы клиентов об экскурсиях
Уведомление	notification	/notifications	Уведомления пользователям

Таблица 1 – Соответствие классов модели ресурсам API

1.4. Принципы проектирования API

При проектировании приняты следующие соглашения:

– **Ресурсно-ориентированные URL**: существительные во множественном числе (/routes, /excursions, /contracts), идентификатор — в path-параметре (/contracts/{contractId}). Для действий, не сводимых к CRUD, используются ресурсы действий: POST /excursions/{id}/start, /finish, /contracts/{id}/cancel, /payments/{id}/refund, /contracts/{id}/close.

– **HTTP-методы**: GET — чтение, POST — создание и действия, PATCH — частичное обновление, PUT — полная замена (для вложенных точек), DELETE — удаление или деактивация ресурса.

– **Коды состояния**: 200/201/204 — успех; 400 — некорректный запрос; 401 — нет токена; 403 — недостаточно прав; 404 — ресурс не найден; 409 — конфликт состояния; 422 — ошибка валидации; 429 — превышен лимит запросов; 500 — внутренняя ошибка.

– **Единый формат ошибок** — Problem Details (RFC 7807): type, title, status, detail, instance; Content-Type: application/problem+json.

– **Пагинация и фильтрация**: page/limit (не более 100), параметры search, sort, dateFrom/dateTo, status; метаданные ответа — page, limit, total, pages.

– **Аутентификация**: Bearer JWT для сотрудников; X-Client-Token для клиента. Передача данных — только по HTTPS.

– **Идемпотентность**: POST /contracts и POST /contracts/{id}/payments принимают заголовок Idempotency-Key; повторный запрос с тем же ключом не создаёт дубль.

– **Форматы значений**: даты и время — ISO 8601 с часовым поясом; денежные суммы — число с двумя знаками после запятой (RUB); перечисления — строковые enum; идентификаторы — UUID.

Для ключевых ресурсов (маршруты, точки, экскурсии, транспорт, гиды) поддерживаются все CRUD-операции: просмотр (GET), добавление (POST), обновление (PATCH/PUT) и удаление (DELETE). Для договоров, платежей, билетов и отчётов дополнительно предусмотрены операции изменения состояния и ресурсы действий (оплата, отмена, возврат, начало и завершение экскурсии, отметка явки).

2. Предварительное проектирование REST API

На этапе предварительного проектирования определены ресурсы API, назначены URL, выбраны действия и описаны входные и выходные данные. Все цели API привязаны к вариантам использования UC-01–UC-32 из предыдущей лабораторной работы.

2.1. Таблица целей API

Таблица 2 содержит цели API, сформулированные с точки зрения потребителя. Для каждой цели указаны пользователь, действие, этапы, входные данные (источник) и выходные данные (способ использования).

№	Пользователи (Кто)	Действия (Что)	Этапы (Как)	Входные данные (источник)	Выходные данные (использование)	Цели
1	Клиент	Купить экскурсию	Найти экскурсию	Параметры поиска (клиент); каталог (сервис каталога)	Список маршрутов и экскурсий	Найти экскурсии в каталоге (UC-01, UC-02)
2	Клиент	Купить экскурсию	Посмотреть маршрут	routeId / excursionId (из каталога)	Описание, точки, расписание	Получить детали маршрута (UC-03)
3	Клиент	Купить экскурсию	Оформить договор	excursionId, данные участников, согласие (клиент)	Договор, электронные билеты, клиентский токен	Оформить договор на экскурсию (UC-04)
4	Клиент	Купить экскурсию	Оплатить	contractId, способ оплаты (клиент)	Платёж, статус оплаты	Провести платёж по договору (UC-08)
5	Клиент	Получить документы	Получить пакет	contractId (из договора)	Договор, счёт, билеты (PDF/QR)	Получить пакет документов (UC-05, UC-07)
6	Клиент	Управлять заказами	Посмотреть историю	Клиентский токен	Список договоров и статусов	Просмотреть историю бронирований (UC-09)
7	Клиент	Управлять заказами	Отказаться от договора	contractId, причина (клиент)	Статус отмены и возврата	Инициировать отказ от договора (UC-10)
8	Клиент	Оценить экскурсию	Оставить отзыв	excursionId, рейтинг, комментарий (клиент)	Отзыв	Оставить отзыв об экскурсии (UC-11)
9	Гид	Работать	Войти в систему	Логин, пароль (гид)	JWT, роль	Авторизоваться в системе (UC-32)

№	Пользователи (Кто)	Действия (Что)	Этапы (Как)	Входные данные (источник)	Выходные данные (использование)	Цели
10	Гид	Работать	Посмотреть расписание	Период (гид)	Список назначенных экскурсий	Просмотреть расписание (UC-29)
11	Гид	Работать	Скачать ведомость	excursionId (гид)	PDF посадочной ведомости и маршрутного листа	Скачать ведомость (UC-30)
12	Гид	Работать	Управлять занятостью	Дата, статус (гид)	Статус занятости	Управлять статусом занятости (UC-28)
13	Гид	Провести экскурсию	Начать, отметить явку, завершить	excursionId, ticketId, attended (гид)	Статусы экскурсии и билетов	Провести экскурсию (UC-24, UC-25)
14	Гид	Отчитаться	Сформировать отчёт	excursionId, текст, фактическая явка (гид)	Отчёт о проведении	Сформировать отчёт (UC-26)
15	Руководитель	Управлять каталогом	CRUD маршрутов и экскурсий	Данные каталога (руководитель)	Маршрут, экскурсия	Управлять каталогом (UC-18)
16	Руководитель	Управлять автопарком	CRUD транспорта	Данные транспорта (руководитель)	Транспорт	Управлять автопарком (UC-19)
17	Руководитель	Управлять гидами	CRUD учётных записей	Данные гида (руководитель)	Гид, статус доступа	Управлять учётными записями гидов (UC-20)
18	Руководитель	Контролировать ресурсы	Мониторинг занятости	Дата (руководитель)	Занятость гидов и транспорта, логи ошибок	Мониторить ресурсы (UC-12)
19	Руководитель	Аналитика	Сформировать сводный отчёт	Период (руководитель)	Сводный аналитический отчёт	Формировать отчёты (UC-14)
20	Руководитель	Работать с клиентами	Найти клиента	Телефон или номер договора (руководитель)	Карточка клиента	Вести поиск и учёт клиентов (UC-21)
21	Руководитель	Финансовый контроль	Открыть реестр	Статус, период (руководитель)	Список договоров и платежей, итоговые суммы	Контролировать реестр (UC-22)
22	Руководитель	Финансовый контроль	Оформить возврат	paymentId / contractId (руководитель)	Статус возврата или закрытия	Инициировать возврат средств (UC-23)

Таблица 2 – Цели API

2.2. Устранение влияния поставщика при проектировании API

Проектирование выполнено с точки зрения потребителя API, а не поставщика. В таблице 3 показано, как исключено влияние внутренних данных, кода, архитектуры и организации поставщика.

Влияние	**Результаты изменения**
1. Данные	API не повторяет структуру базы данных. Наружу отдаются DTO с публичными идентификаторами (UUID), осмысленными именами и вложенными объектами; внутренние ключи, служебные поля и таблицы связей скрыты. Справочные значения ограничены enum-перечислениями (статусы договора, экскурсии и платежа, тип передвижения, специализация гида).
2. Код и бизнес-логика	Действия сформулированы как цели потребителя (оформить договор, провести платёж, отметить явку), а не как вызовы внутренних методов. Расчёт стоимости, проверка свободных мест и правила оплаты скрыты за ресурсами; клиент не знает последовательности обработки. Для не-CRUD действий используются ресурсы действий: POST /start, /finish, /cancel, /refund, /close.
3. Архитектура ПО	API — фасад над доменными сервисами; клиент не зависит от внутренней архитектуры (монолит или микросервисы, СУБД, очереди). Версионирование /api/v1, единый формат ошибок Problem Details, единые пагинация и фильтры. Внешние системы (банковский эквайринг, SMTP) инкапсулированы за API.
4. Организация	Роли API (Клиент, Гид, Руководитель) соответствуют бизнес-ролям, а не отделам компании; права проверяются на уровне эндпоинтов. Термины согласованы с моделью предметной области и диаграммами UC; спецификация OpenAPI служит единым контрактом для бизнеса, аналитиков и разработчиков.

Таблица 3 – Устранение влияния поставщика

2.3. Соотнесение целей API с диаграммой вариантов использования

В таблице 4 цели API сопоставлены с целевыми вариантами использования и их расширениями/включениями. Формат соответствует диаграммам UC, разработанным ранее: указаны субъект, целевой UC и связанные UC с типом отношения.

Субъект (Actor)	Целевой вариант использования (UC)	Расширения и включения (Include, Extend)
Клиент	UC-01 Просмотреть каталог маршрутов и экскурсий	UC-02 «Отфильтровать и отсортировать экскурсии» — расширение; UC-03 «Просмотреть детальное описание маршрута» — расширение
Клиент	UC-04 Оформить договор на экскурсию	UC-05 «Получить пакет документов» — включение; UC-08 «Провести платёж по договору» — включение
Клиент	UC-05 Получить пакет документов (договор, счёт, билеты)	UC-06 «Получить копию пакета документов на Email» — обобщение; UC-07 «Скачать документы и электронные билеты (PDF/QR)» — обобщение
Клиент	UC-08 Провести платёж по договору	—
Клиент	UC-09 Просмотреть историю бронирований (договоров)	UC-10 «Инициировать отказ от договора» — расширение
Клиент	UC-11 Оставить отзыв об экскурсии	—
Гид	UC-24 Провести экскурсию (смена статуса «начата»/«завершена»)	UC-25 «Проконтролировать посадку» — включение; UC-26 «Сформировать и отправить отчет» — включение; UC-27 «Зафиксировать инцидент (ЧП)» — расширение UC-26
Гид	UC-26 Сформировать и отправить отчет о проведении	UC-27 «Зафиксировать инцидент (ЧП) и прикрепить фото» — расширение
Гид	UC-29 Просмотреть расписание и детали назначенной экскурсии	UC-32 «Авторизоваться в системе» — включение; UC-30 «Скачать посадочную ведомость и маршрутный лист» — расширение
Гид	UC-28 Управлять статусом своей занятости	—
Гид	UC-31 Редактировать контактные данные профиля	—
Руководитель	UC-12 Мониторить занятость ресурсов и логи ошибок распределения	UC-13 «Вручную переназначить гида или транспорт (форс-мажор)» — расширение
Руководитель	UC-14 Формировать сводные аналитические отчеты	UC-15 «Просматривать отчеты гидов о проведенных экскурсиях» — включение; UC-16 «Экспортировать отчет в формат Excel» — расширение
Руководитель	UC-17 Управлять справочниками ресурсов и каталогом	UC-18, UC-19, UC-20 — обобщение; UC-32 «Авторизоваться в системе» — включение
Руководитель	UC-21 Вести поиск и учет клиентов	—
Руководитель	UC-22 Контролировать реестр договоров и платежей	UC-32 «Авторизоваться в системе» — включение; UC-23 «Инициировать возврат средств и закрытие сделок» — расширение

Субъект (Actor)	Целевой вариант использования (UC)	Расширения и включения (Include, Extend)
Сотрудник бюро	UC-32 Авторизоваться в системе	Наследуется актёрами «Гид» и «Руководитель (Администратор)»

Таблица 4 – Соответствие целей API и вариантов использования

2.4. Окончательная таблица целей API

После сопоставления с диаграммой UC таблица целей дополнена целями, которые отсутствовали в первоначальном варианте: копия пакета документов на email (UC-06), фиксация инцидента и фотографий (UC-27), редактирование профиля гида (UC-31), переназначение ресурсов (UC-13), просмотр отчётов гидов (UC-15) и экспорт отчёта в Excel (UC-16). Окончательный перечень целей приведён в таблице 5.

№	Пользователь	Цель API	UC	Входные данные	Результат	Статус
1	Клиент	Найти экскурсии в каталоге	UC-01, UC-02	Фильтры, сортировка, страница	Список экскурсий	исходная
2	Клиент	Получить детали маршрута	UC-03	routeId, excursionId	Маршрут, точки, расписание	исходная
3	Клиент	Оформить договор	UC-04	excursionId, участники, согласие	Договор, билеты, токен	исходная
4	Клиент	Провести платёж	UC-08	contractId, способ оплаты	Платёж, статус	исходная
5	Клиент	Получить пакет документов	UC-05, UC-07	contractId	PDF-документы, билеты	исходная
6	Клиент	Просмотреть историю	UC-09	Клиентский токен	Список договоров	исходная
7	Клиент	Отказаться от договора	UC-10	contractId, причина	Статус отмены	исходная
8	Клиент	Оставить отзыв	UC-11	excursionId, рейтинг, текст	Отзыв	исходная
9	Гид	Авторизоваться	UC-32	Логин, пароль	JWT, роль	исходная
10	Гид	Просмотреть расписание	UC-29	Период	Список экскурсий	исходная
11	Гид	Скачать ведомость	UC-30	excursionId	PDF ведомости	исходная
12	Гид	Управлять занятостью	UC-28	Дата, статус	Статус занятости	исходная
13	Гид	Провести экскурсию	UC-24, UC-25	excursionId, ticketId, attended	Статусы экскурсии и билетов	исходная

№	Пользователь	Цель API	UC	Входные данные	Результат	Статус
14	Гид	Сформировать отчёт	UC-26	excursionId, текст, явка	Отчёт	исходная
15	Руководитель	Управлять каталогом	UC-18	Данные маршрута/экскурсии	Маршрут/экскурсия	исходная
16	Руководитель	Управлять автопарком	UC-19	Данные транспорта	Транспорт	исходная
17	Руководитель	Управлять учётными записями гидов	UC-20	Данные гида	Гид, доступ	исходная
18	Руководитель	Мониторить ресурсы	UC-12	Дата	Занятость, логи ошибок	исходная
19	Руководитель	Формировать сводные отчёты	UC-14	Период	Сводный отчёт	исходная
20	Руководитель	Вести поиск клиентов	UC-21	Телефон, номер договора	Карточка клиента	исходная
21	Руководитель	Контролировать реестр	UC-22	Статус, период	Договоры, платежи, суммы	исходная
22	Руководитель	Инициировать возврат	UC-23	paymentId, contractId	Статус возврата	исходная
23	Клиент	Получить копию пакета на email	UC-06	contractId, email	Статус отправки	добавлена
24	Гид	Зафиксировать инцидент и фото	UC-27	reportId, описание, файлы	Отчёт с ЧП и фото	добавлена
25	Гид	Редактировать контактные данные	UC-31	Телефон, email	Профиль	добавлена
26	Руководитель	Переназначить гида или транспорт	UC-13	excursionId, guideId, transportId	Назначение	добавлена
27	Руководитель	Просмотреть отчёты гидов	UC-15	Период, guideId	Отчёты гидов	добавлена
28	Руководитель	Экспортировать отчёт в Excel	UC-16	Период	XLSX-файл	добавлена

Таблица 5 – Окончательная таблица целей API

2.5. Сопоставление вариантов использования с маршрутами и конечными точками

Маршрут (Route) — URL, направляющий запрос к конечным точкам; конечная точка (Endpoint) — обращение к маршруту конкретным HTTP-методом. В таблице 6 каждый UC сопоставлен маршрутам и HTTP-методам. Базовый префикс всех маршрутов — /api/v1.

Субъект (Actor)	Вариант использования (UC)	Маршрут (Route)	HTTP-метод
Клиент	UC-01 Просмотреть каталог	/routes; /excursions	GET
Клиент	UC-02 Отфильтровать и отсортировать	/routes?search={q} &sort={field}&page={n} &limit={m}; /excursions?dateFrom={d} &dateTo={d}&routeId={id}	GET
Клиент	UC-03 Детали маршрута	/routes/{routeId}; /routes/{routeId}/points; /excursions/{excursionId}	GET
Клиент	UC-04 Оформить договор	/contracts	POST
Клиент	UC-05 Получить пакет документов	/contracts/{contractId}/ documents; /contracts/{contractId}/ tickets	GET
Клиент	UC-06 Копия пакета на email	/contracts/{contractId}/ documents/email	POST
Клиент	UC-07 Скачать документы и билеты	/contracts/{contractId}/ documents/{documentId}; /contracts/{contractId}/ tickets/{ticketId}/pdf	GET
Клиент	UC-08 Провести платёж	/contracts/{contractId}/ payments; /payments/{paymentId}	POST, GET
Клиент	UC-09 История бронирований	/contracts; /contracts/{contractId}	GET
Клиент	UC-10 Отказ от договора	/contracts/{contractId}/ cancel	POST
Клиент	UC-11 Оставить отзыв	/excursions/{excursionId}/ reviews	POST, GET
Гид	UC-32 Авторизоваться	/auth/login; /auth/refresh; /auth/me; /auth/logout	POST, GET
Гид	UC-29 Расписание и детали	/guides/me/schedule; /guides/me/excursions/ {excursionId}	GET
Гид	UC-30 Скачать ведомость	/excursions/{excursionId}/ boarding-list; /excursions/{excursionId}/ boarding-list.pdf	GET
Гид	UC-28 Управлять занятостью	/guides/me/availability	PATCH
Гид	UC-24 Провести экскурсию	/excursions/{excursionId}/ start; /excursions/{excursionId}/ finish	POST
Гид	UC-25 Посадка и явка	/excursions/{excursionId}/ tickets; /tickets/{ticketId}	GET, PATCH
Гид	UC-26 Сформировать отчёт	/excursions/{excursionId}/ reports; /reports/{reportId}	POST, GET, PATCH

Субъект (Actor)	Вариант использования (UC)	Маршрут (Route)	HTTP-метод
Гид	UC-27 Инцидент и фото	/reports/{reportId}; /reports/{reportId}/photos; /reports/{reportId}/photos/{photoId}	PATCH, POST, DELETE
Гид	UC-31 Профиль	/guides/me	GET, PATCH
Руководитель	UC-12 Мониторинг ресурсов	/resources/availability?date={d}; /resource-errors?date={d}	GET
Руководитель	UC-13 Переназначить ресурсы	/excursions/{excursionId}/assignments	POST, PATCH
Руководитель	UC-14 Сводные отчёты	/analytics/summary?dateFrom={d}&dateTo={d}; /analytics/revenue?dateFrom={d}&dateTo={d}	GET
Руководитель	UC-15 Отчёты гидов	/reports/guides?dateFrom={d}&dateTo={d}&guideId={id}	GET
Руководитель	UC-16 Экспорт в Excel	/analytics/export.xlsx?dateFrom={d}&dateTo={d}	GET
Руководитель	UC-17 Управлять справочниками	— (реализуется через UC-18–UC-20)	—
Руководитель	UC-18 Управлять каталогом	/routes; /routes/{routeId}; /routes/{routeId}/points; /routes/{routeId}/points/{pointId}; /excursions; /excursions/{excursionId}	POST, PATCH, DELETE
Руководитель	UC-19 Управлять автопарком	/transports; /transports/{transportId}	GET, POST, PATCH, DELETE
Руководитель	UC-20 Управлять учётными записями гидов	/guides; /guides/{guideId}; /guides/{guideId}/access	GET, POST, PATCH, DELETE
Руководитель	UC-21 Поиск и учёт клиентов	/clients?search={q}&phone={p}&contractNumber={n}; /clients/{clientId}	GET
Руководитель	UC-22 Реестр договоров и платежей	/contracts?status={s}&dateFrom={d}&dateTo={d}; /payments?status={s}; /payments/summary?dateFrom={d}&dateTo={d}	GET
Руководитель	UC-23 Возврат и закрытие	/payments/{paymentId}/refund; /contracts/{contractId}/close	POST

Таблица 6 – Сопоставление вариантов использования с маршрутами и конечными точками

2.6. Примеры запросов и ответов

Ниже приведены примеры вызовов API для ключевых целей: оформление договора, проведение платежа, отметка явки и обработка ошибки. Примеры показывают используемые методы, заголовки, коды состояния и форматы данных.

Пример 1. Оформление договора (UC-04)

```
POST /api/v1/contracts HTTP/1.1
Host: api.excursion-bureau.ru
Content-Type: application/json
Idempotency-Key: 6f1c2b0e-7d3a-4c9e-9f1a-2b3c4d5e6f70

{
  "excursionId": "e3b0c442-98fc-4c14-9b9f-1f5d3a7b8c21",
  "participants": [
    {
      "lastName": "Петров", "firstName": "Пётр", "middleName": "Петрович",
      "passportSeries": "9701", "passportNumber": "123456"
    }
  ],
  "contactEmail": "petrov@example.ru",
  "contactPhone": "+79001234567",
  "consentPersonalData": true
}

HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/v1/contracts/8a7b6c5d-4e3f-4a2b-9c1d-0e9f8a7b6c5d

{
  "contract": {
    "id": "8a7b6c5d-4e3f-4a2b-9c1d-0e9f8a7b6c5d",
    "number": "Д-2026-001234",
    "excursionId": "e3b0c442-98fc-4c14-9b9f-1f5d3a7b8c21",
    "totalPrice": 1500.00,
    "status": "pending_payment",
    "paymentDeadline": "2026-11-15T10:10:00+03:00"
  },
  "clientToken": "ct_7f3a9c2e5b1d"
}
```

Пример 2. Проведение платежа (UC-08)

```
POST /api/v1/contracts/8a7b6c5d-4e3f-4a2b-9c1d-0e9f8a7b6c5d/payments HTTP/1.1
Host: api.excursion-bureau.ru
X-Client-Token: ct_7f3a9c2e5b1d
Content-Type: application/json
Idempotency-Key: 0b9c8d7e-6f5a-4b3c-2d1e-0f9a8b7c6d5e

{
  "method": "card",
  "returnUrl": "https://excursion-bureau.ru/payment/result"
}

HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": "b1c2d3e4-5f6a-7b8c-9d0e-1f2a3b4c5d6e",
  "contractId": "8a7b6c5d-4e3f-4a2b-9c1d-0e9f8a7b6c5d",
  "amount": 1500.00,
  "method": "card",
  "status": "pending",
  "dueDate": "2026-11-15T10:10:00+03:00"
}
```

Пример 3. Отметка явки участника (UC-25)

```
PATCH /api/v1/tickets/9f8e7d6c-5b4a-3c2d-1e0f-9a8b7c6d5e4f HTTP/1.1
Host: api.excursion-bureau.ru
Authorization: Bearer <JWT гида>
Content-Type: application/json

{ "attended": true }

HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": "9f8e7d6c-5b4a-3c2d-1e0f-9a8b7c6d5e4f",
  "participantName": "Петров Пётр Петрович",
  "attended": true,
  "status": "used"
}
```

Пример 4. Конфликт состояния при отмене оплаченного договора

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "https://api.excursion-bureau.ru/problems/contract-conflict",
  "title": "Конфликт состояния договора",
  "status": 409,
  "detail": "Договор уже оплачен; для отмены требуется возврат средств",
  "instance": "/api/v1/contracts/8a7b6c5d-4e3f-4a2b-9c1d-0e9f8a7b6c5d/cancel"
}
```

2.7. Структура данных ресурсов «Экскурсия» (Product) и «Гид» (Provider)

В предметной области «товаром» является экскурсия (конкретное проведение маршрута), а «поставщиком» услуги — гид, который может быть как штатным сотрудником, так и внешним подрядчиком. Структура данных этих ресурсов приведена в таблице 7.

Ресурс	Поле	Описание свойства	Тип данных	Обязательно
excursion	id	Идентификатор экскурсии	string (UUID)	да
excursion	routeId	Идентификатор маршрута	string (UUID)	да
excursion	routeName	Название маршрута (только чтение)	string	да
excursion	guide	Поставщик услуги (объект guide)	object	да
excursion	transportId	Идентификатор транспортного средства	string (UUID)	нет
excursion	startDateTime	Дата и время начала	string (date-time)	да
excursion	endDateTime	Дата и время окончания	string (date-time)	да
excursion	price	Стоимость участия, ₽	number	да
excursion	maxGroupSize	Максимальный размер группы	integer	да

Ресурс	Поле	Описание свойства	Тип данных	Обязательно
excursion	availableSeats	Свободные места (только чтение)	integer	да
excursion	status	Статус: scheduled, started, completed, closed, cancelled	string (enum)	да
excursion	createdAt	Дата создания	string (date-time)	нет
guide	id	Идентификатор гида	string (UUID)	да
guide	lastName	Фамилия	string	да
guide	firstName	Имя	string	да
guide	middleName	Отчество	string	нет
guide	phone	Телефон	string	да
guide	email	Электронная почта	string (email)	да
guide	specialization	Специализация: city, museum, mountain	string (enum)	да
guide	status	Статус занятости: available, busy	string (enum)	да
guide	externalContractor	Внешний подрядчик	boolean	нет
guide	accessActive	Доступ активен	boolean	да
guide	createdAt	Дата создания	string (date-time)	нет

Таблица 7 – Структура данных ресурсов «Экскурсия» и «Гид»

Пример представления экскурсии в формате JSON (вложенный объект guide):

```
{
  "id": "e3b0c442-98fc-4c14-9b9f-1f5d3a7b8c21",
  "routeId": "a1d2e3f4-5678-4abc-9def-0123456789ab",
  "routeName": "Чебоксары исторические",
  "guide": {
    "id": "5f1c2d3e-4a5b-6c7d-8e9f-0a1b2c3d4e5f",
    "lastName": "Иванов",
    "firstName": "Иван",
    "middleName": "Иванович",
    "phone": "+79001234567",
    "email": "ivanov@example.ru",
    "specialization": "city",
    "status": "available",
    "externalContractor": false,
    "accessActive": true
  },
  "startDateTime": "2026-11-15T10:00:00+03:00",
  "endDateTime": "2026-11-15T13:00:00+03:00",
  "price": 1500.00,
  "maxGroupSize": 20,
  "availableSeats": 12,
  "status": "scheduled"
}
```

Листинг 1 – Пример ресурса «Экскурсия» в формате JSON

2.8. Структура данных ресурса «Маршрут» и вложенного ресурса «Точка»

Маршрут — каталожная сущность, на основе которой создаются экскурсии; он содержит вложенный массив точек. Структура данных приведена в таблице 8.

Ресурс	Поле	Описание свойства	Тип данных	Обязательно
route	id	Идентификатор маршрута	string (UUID)	да
route	name	Название маршрута	string	да
route	description	Описание	string	нет
route	durationHours	Продолжительность, ч	integer	да
route	basePrice	Базовая стоимость, ₽	number	да
route	maxGroupSize	Максимальный размер группы	integer	да
route	ageRestriction	Возрастное ограничение (например, 6+)	string	нет
route	movementType	Тип передвижения: walking, bus	string (enum)	да
route	active	Признак активности	boolean	да
route	createdAt	Дата создания	string (date-time)	нет
route	points	Массив точек маршрута	array<Point>	да
point	id	Идентификатор точки	string (UUID)	да
point	name	Название точки	string	да
point	description	Описание	string	нет
point	country	Страна	string	да
point	locality	Населённый пункт	string	да
point	latitude	Широта	number (float)	да
point	longitude	Долгота	number (float)	да
point	order	Порядок посещения	integer	да

Таблица 8 – Структура данных ресурсов «Маршрут» и «Точка»

2.9. Проверка параметров и полноты данных

Для каждого действия проверено, что потребитель может предоставить все необходимые параметры и получить требуемые результаты. Серверные поля (id, createdAt, availableSeats, статусы) не передаются клиентом — они

генерируются или вычисляются сервером. Источники входных параметров приведены в таблице 9.

Параметр	Где потребитель получает значение	Пример использования
excursionId	GET /excursions (каталог)	Оформление договора
routeId	GET /routes (каталог)	Просмотр маршрута и создание экскурсии
contractId	POST /contracts (ответ contract.id)	Оплата, документы, отмена
paymentId	POST /contracts/{contractId}/payments	Проверка статуса, возврат
ticketId	GET /contracts/{contractId}/tickets	Отметка явки гидом
reportId	POST /excursions/{excursionId}/reports	Дополнение отчёта, фото
guideId, transportId	GET /resources/availability?date=	Назначение и переназначение

Таблица 9 – Источники входных параметров

Если бы для действия требовались данные, недоступные потребителю, это указывало бы на пропущенную цель или на влияние внутренней структуры поставщика. В спроектированном API таких разрывов нет: все обязательные параметры либо вводятся пользователем, либо получаются из ответов предыдущих запросов.

3. Спецификация OpenAPI

Предварительное проектирование завершено машинно-читаемой спецификацией OpenAPI 3.0.3. Спецификация сохранена в файле **7_openapi.yaml** и может быть открыта в Swagger Editor, Postman, Stoplight или использована для генерации клиентского и серверного кода.

3.1. Состав спецификации

Раздел	Содержание
info	Название, версия 1.0.0, описание, контакты
servers	Продуктивный и тестовый URL (https://…/api/v1)
tags	8 групп операций: Auth, Catalog, Contracts, Payments, Documents, Guide, Manager, Analytics
securitySchemes	bearerAuth (JWT для сотрудников), clientToken (X-Client-Token для клиента)

Раздел	Содержание
paths	55 путей: CRUD ресурсов, действия (start, finish, cancel, refund, close), документы и аналитика
components.schemas	32 схемы данных: Route, Point, Excursion, Guide, Transport, Contract, Payment, Ticket, Review, Report и др.
components.responses	Общие ответы 401, 403, 404, 409, 422 в формате Problem Details

Таблица 10 – Состав спецификации OpenAPI

3.2. Фрагмент спецификации

Ниже приведён фрагмент спецификации: описание договора и операции оформления договора.

```
openapi: 3.0.3
info:
  title: API информационной системы «Экскурсионное бюро»
  version: 1.0.0
servers:
  - url: https://api.excursion-bureau.ru/api/v1
security:
  - bearerAuth: []
paths:
  /contracts:
    post:
      tags: [Contracts]
      summary: Оформить договор на экскурсию (UC-04)
      security: []
      parameters:
        - name: Idempotency-Key
          in: header
          schema: { type: string }
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ContractInput'
      responses:
        '201':
          description: Договор создан; возвращается клиентский токен
          content:
            application/json:
              schema:
                type: object
                properties:
                  contract: { $ref: '#/components/schemas/Contract' }
                  clientToken: { type: string }
        '409':
          $ref: '#/components/responses/Conflict'
components:
  securitySchemes:
    bearerAuth: { type: http, scheme: bearer, bearerFormat: JWT }
    clientToken: { type: apiKey, in: header, name: X-Client-Token }
  schemas:
    ContractInput:
      type: object
      required: [excursionId, participants, contactEmail, consentPersonalData]
      properties:
        excursionId: { type: string, format: uuid }
        participants:
          type: array
```

```
      minItems: 1
      maxItems: 20
      items: { $ref: '#/components/schemas/Participant' }
    contactEmail: { type: string, format: email }
    consentPersonalData: { type: boolean }
```

Листинг 2 – Фрагмент спецификации OpenAPI (файл 7_openapi.yaml)

Полная спецификация включает описание всех ресурсов, параметров, кодов ответов и примеров. Ссылки между разделами проверены; YAML-документ успешно прошёл валидацию openapi-spec-validator (результат: OK) и может быть открыт в Swagger Editor, Postman или Stoplight.

4. Ответы на контрольные вопросы

1. Что еще, кроме REST API, можно использовать для организации связи между приложениями?

SOAP, GraphQL, gRPC, WebSocket, JSON-RPC/XML-RPC, брокеры сообщений (RabbitMQ, Kafka), webhooks и очереди задач, а также файловый обмен и общие базы данных.

2. Какие типы API существуют?

По назначению и аудитории: публичные, партнёрские, внутренние (private). По стилю и протоколу: REST, SOAP, GraphQL, gRPC, WebSocket, RPC. По уровню: библиотечные, операционные, веб-API.

3. Приведите примеры использования API.

Платёжные API (эквайринг, возвраты), картографические и геолокационные сервисы, погодные API, мессенджеры и SMS, CRM/ERP-интеграции, маркетплейсы и логистика, аутентификация через внешние сервисы (OAuth).

4. Что такое спецификация API?

Машинно-читаемый контракт API (например, OpenAPI/Swagger), описывающий серверы, эндпоинты, HTTP-методы, параметры, схемы запросов и ответов, аутентификацию и ошибки. Спецификация служит

единым источником истины для документации, генерации кода и тестирования.

5. Что такое конечная точка API?

Это конкретный адрес (URI) вместе с HTTP-методом, по которому клиент отправляет запрос и получает ответ. Конечная точка является частью маршрута ресурса; у одного ресурса может быть несколько конечных точек.

6. Какие задачи решаются на этапе предварительного проектирования API?

Определение ресурсов и их связей, назначение URL, выбор действий (HTTP-методов), определение входных параметров и результатов, проектирование структур данных, проверка полноты параметров, устранение влияния поставщика и привязка целей к UC.

7. Какие команды HTTP являются идемпотентными?

GET, HEAD, PUT, DELETE, OPTIONS, TRACE. Повторный вызов с теми же данными даёт тот же результат состояния. POST и PATCH в общем случае неидемпотентны; для платежей идемпотентность обеспечивается заголовком Idempotency-Key.

8. Существует ли клиентский веб-API?

Да. В браузере доступны клиентские Web API: Fetch, DOM, Web Storage, Geolocation, Notifications, Canvas и др. Они предоставляются браузером, а не сервером; серверный веб-API — это удалённый API, доступный по HTTP.

9. Из чего состоит серверный веб-API?

Из набора конечных точек и маршрутов, HTTP-методов, форматов запросов и ответов (JSON/XML), аутентификации и авторизации, бизнес-логики, слоя доступа к данным, обработки ошибок, журналирования и документации (спецификации).

10. Как в REST API передаются данные на сервер?

Через параметры пути (/contracts/{contractId}), строку запроса (GET ? search=…&page=…), заголовки (Authorization, X-Client-Token, Idempotency-Key) и тело запроса (JSON для POST/PUT/PATCH, multipart/form-data для файлов).

11. Из каких шагов складывается обработка API-запросов?

Маршрутизация запроса, аутентификация и проверка прав, валидация параметров и тела, выполнение бизнес-логики, обращение к данным и внешним сервисам, формирование ответа и кода состояния, журналирование и возврат ответа клиенту.

12. Продолжите предложение «При разработке API важно иметь глубокие и точные знания о том, …»

…кто может использовать API; что он может делать; как он это делает; что ему нужно для этого; что он получает взамен. Эти сведения образуют основу целей API и позволяют спроектировать интерфейс с точки зрения потребителя.

5. Заключение

В ходе лабораторной работы выполнено предварительное проектирование REST API информационной системы «Экскурсионное бюро»:

– определены границы API, пользователи и роли, ресурсы и их связь с классами модели предметной области;

– составлена таблица целей API с указанием пользователей, действий, этапов, входных и выходных данных;

– устранено влияние поставщика по четырём направлениям: данные, код и бизнес-логика, архитектура ПО, организация;

– цели API сопоставлены с диаграммой вариантов использования UC-01–UC-32, сформирована окончательная таблица целей;

– варианты использования сопоставлены с маршрутами и конечными точками (HTTP-методы, path- и query-параметры);

– спроектированы структуры данных ресурсов «Экскурсия» (Product), «Гид» (Provider), «Маршрут» и «Точка», проверена полнота входных параметров;

– разработана и проверена спецификация OpenAPI 3.0.3 (файл 7_openapi.yaml, 55 путей, 32 схемы), пригодная для документирования, генерации кода и тестирования.

Полученный API покрывает все основные действия с ресурсами (просмотр, создание, обновление, удаление, действия) и может быть использован клиентскими приложениями клиента, гида и руководителя.